

参考：

- 《Java 7之基础 - 强引用、弱引用、软引用、虚引用》 [链接](#)

JVM 内存模型

JVM 在程序运行过程中，会将内存划分为若干个数据区域

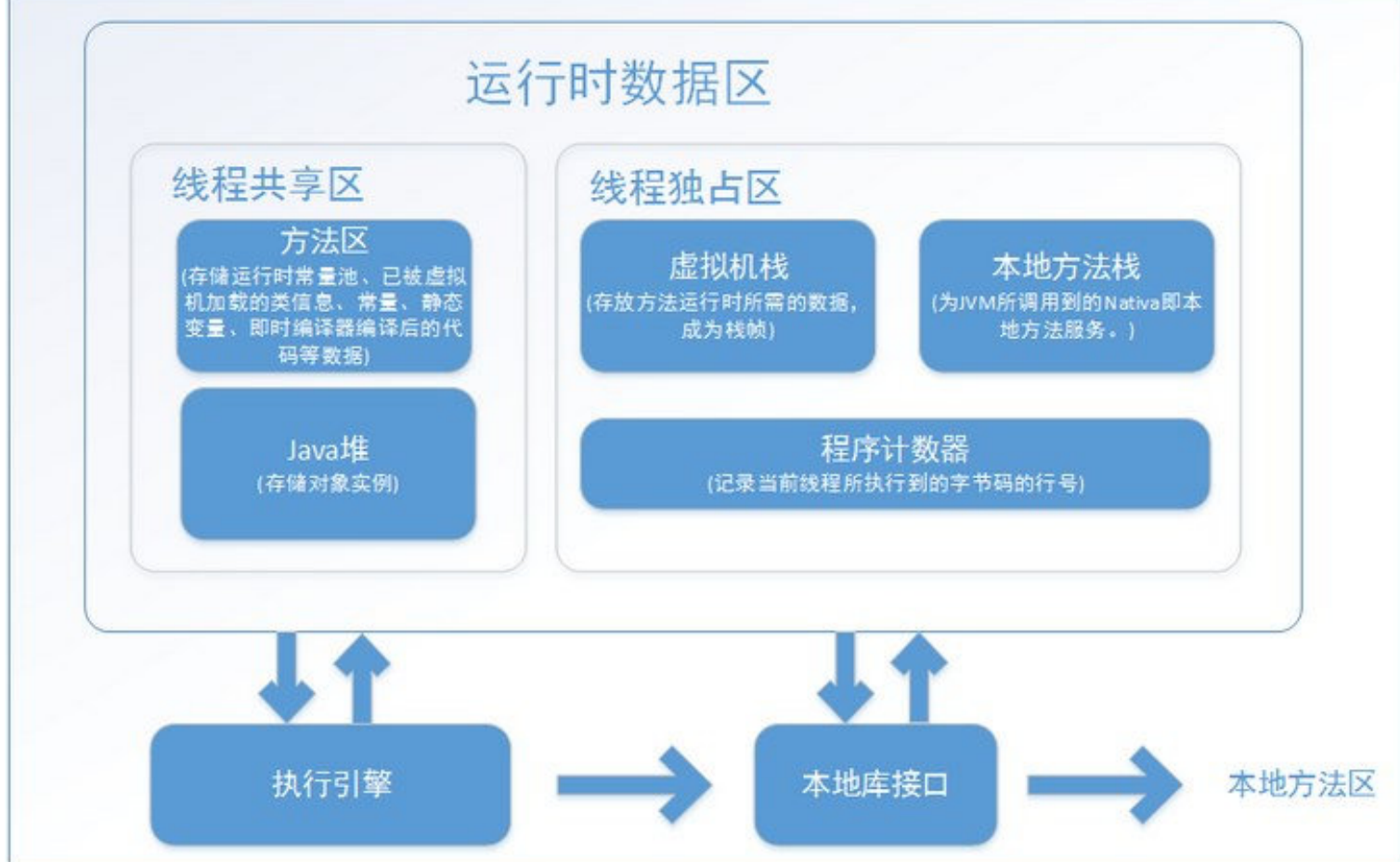
- 内存区域：方法区、堆、栈（虚拟机栈、本地方法栈）、程序计数器
- 这些区域，有的是线程共享的、有的是线程独立的
- 栈：先进后出；队列：先进先出

方法区：存储 类的信息（方法表、属性表）、常量池、静态变量、即时编译器编译后的代码等 运行时数据结构

堆：存储 对象实例 的

栈：存储方法执行的内存模型，以 栈帧 为基本单位，包括 局部变量表、操作数栈、返回值、动态链接 等等

程序计数器：存储当前线程 字节码行号指示器



程序计数器

程序计数器 – Program Counter Register

程序计数器：**当前线程** 执行的 **字节码行号指示器**，标志当前线程执行到哪个位置，是一块较小的内存空间

- 字节码解释器工作时，通过改变计数器的值，来获取下一条需要执行的字节码指令
- 根据 JVM 执行的方法不同，程序计数器记录的值不同：
 - Java 方法，程序计数器记录当前 **字节码指令地址**
 - Native 方法，记录的是 **undefined**

- 分支、循环、跳转、异常处理、线程恢复等功能都需要依赖程序计数器

程序计数器是 **线程私有** 的，每个线程都会拥有一个独立的程序计数器

- 为什么？JVM 是多线程的，每个线程都有独立的指令地址

程序计数器是唯一一个不会出现 `OutOfMemoryError` 情况的区域

栈

栈：存储方法运行时所需要的数据（虚拟机栈为 Java 方法服务，本地方法栈为 Native 方法服务）

1. **本地方法栈**：Native Method Stack，是 **Native 方法** 的内存模型（存储 Native 方法运行时所需要的数据）
2. **Java 虚拟机栈**：Java Virtual Machine Stacks，是 **Java 方法** 执行的内存模型（存储 Java 方法运行时所需要的数据）

栈是 **线程私有** 的，每条线程单独占有一条栈

栈通常会抛出 2 种异常

- 如果栈 **不可动态扩展**：线程请求的深度，大于栈所允许的最大深度，抛出 `StackOverflowError`（栈溢出）
- 如果栈 **可以动态扩展**：扩展时无法申请到足够的内存，抛出

`OutOfMemoryError`（内存溢出）

堆栈 `max_stacks`：它用于保存方法调用过程中的局部变量、中间结果和操作数栈（operand stack）。

栈帧

栈帧：方法运行时的基础数据结构（栈是内存模型），每一个方法在运行的同时都会创建一个栈帧

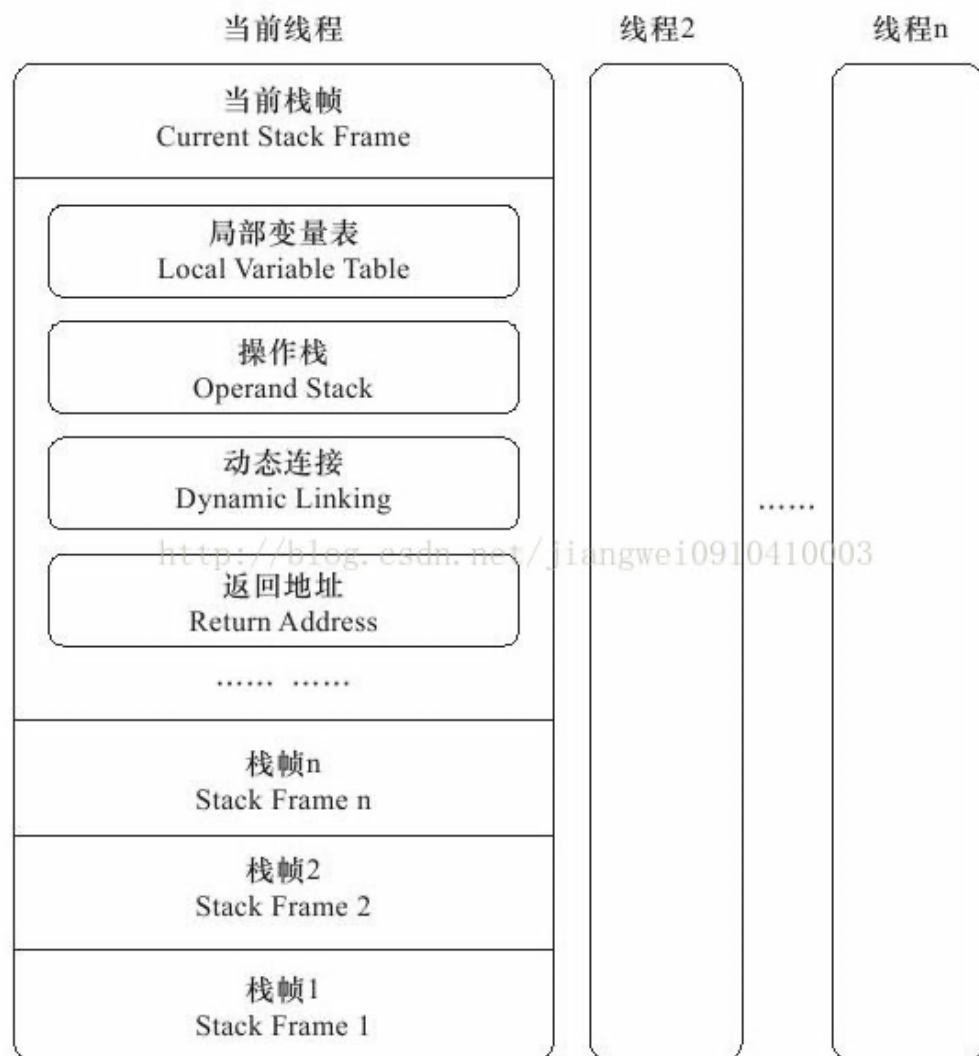
- 方法的调用，就是栈帧从 **入栈到出栈** 的过程
- 入栈（压栈）：方法由方法区的方法表，压入虚拟机栈中，形成一个栈帧

栈帧由 **局部变量表**、**操作数栈**、**返回地址**、**动态链接** 和一些额外的 **附加信息** 组成

栈帧所占用的内存在编译期就已经确定了

- 在编译期就已经确定了 **局部变量表的大小** 和 **操作数栈的深度**，因此栈帧所占用的内存，在程序运行期不会受变量数据的影响

`Thread.currentThread().getStackTrace()` 获取当前方法的栈帧信息



- 局部变量表

局部变量表：用于存放 **方法参数** 和方法体内的 **局部变量**，是一组变量值的存储空间

- 局部变量表的内存空间大小，在 **编译期** 就已经确定了，并存储在方法表的 **Code 属性表** 的 **max_locals** 数据项中，在程序运行过程中不会改变

局部变量表以 **变量槽 Variable Slot** 为最小单位

- 变量槽存储的 **数据类型**

- boolean、char、byte、int、long、float、reference、returnAddress 等数据类型
- 变量槽的 大小
 - 虚拟机并没有明确指出其大小，一般我们认为在 32 位系统中占 32 位
 - double、long 占用 64 位，则会以高位对其的方式连续占据 2 个 Slot 槽，访问的时候只允许同时访问 2 个，不允许单独访问
 - 除了 long 和 double 类型占用 2 个 Slot 外，其他类型均占 1 个 Slot
- 变量槽可以被 复用
 - 如果程序计数器超出了某个变量的作用域，那么该变量占用的 Slot 可以交由其他变量使用
 - 手动置 null，可以加速对象的回收

// 方法执行完后，依然占有 10M 空间

```
public void test() {  
    {  
        byte[] bytes = new byte[1024*1024*10];  
    }  
    System.gc();  
}
```

// 方法执行完后，bytes 所占的 Slot 被 a 复用了

```
public void test() {  
    {  
        byte[] bytes = new byte[1024*1024*10];  
    }
```

```
}  
int a = 1;  
System.gc();  
}
```

局部变量没有零值

- 类变量有个 **准备阶段** 赋初始值，但是局部变量没有该阶段
- 如果定义了一个局部变量，但是没有赋初始值，那么该变量是没法使用的

```
// 下面的代码会报错，因为没有赋初始值  
public void test() {  
    int a;  
    System.out.print(a);  
}
```

- 操作数栈

操作数栈：存储方法中生成的 **字节码指令**，是一个先入后出的栈

- 在方法的执行过程中，生成的字节码指令会进行一个入栈/出栈操作
- 操作数栈的最大深度，在编译的时候存入 **Code 属性表** 的 **max_stacks** 数据项中
- 一般来说，局部变量表越大，对应对操作栈的最大深度就越小（在发生 StackOverflow 的时候可以优化）
- 如何计算操作栈的最大深度：递归调用，手动计算 count 值

- 动态链接

动态链接

- 每个栈帧都包含有指向 **常量池** 中该栈帧所属 **方法的引用**
- 每个方法都包含有指向 **常量池** 中该方法所属 **对象的引用**

- 方法返回地址

方法运行后有 2 种方法可以退出该方法：

- **正常完成出口**：执行引擎遇到任意一个方法返回的字节码指令
- **异常完成出口**：执行引擎遇到一个未处理的异常

堆

Java 堆 - Java Heap

Java 堆：用来存放 **对象实例** 的，是 Java 虚拟机所管理内存中 **最大的一块**

- Java 堆可以是物理上 **不连续** 的内存空间
- Java 堆是垃圾回收器回收的主要区域，也叫 **GC 堆**
(Garbage Collected Heap)

- Java 堆根据垃圾回收机制又可分为新生代、永久代等等

Java 堆是 **线程共享** 的

Java 堆会抛出 OutOfMemoryError 异常

Native 堆 - Native Heap

- Java Heap 有一个上限（上限 VM heap）,
- Native Heap 不受 VM heap 的限制, 只受限于 RAM 内存

方法区

方法区 - Method Area

方法区：用来存储 **类的信息（字段表、方法表、属性表）**、**常量池**、**静态变量**、**方法代码**、即时编译器编译后的代码等 **运行时数据结构**

方法区和 Java 堆表现非常相似, 但是 GC 回收在这块区域是比较少出现的 (类似 **永久代**)

方法区是 **线程共享** 的

方法区会抛出 OutOfMemoryError 异常

在 JDK7 之前，常量、静态变量都存储在 **方法区中（永久代）**

在 JDK7 极其之后，为了避免因为“字符串字面量”的大量存储而造成永久代内存溢出，常量、静态变量都存储到 **堆** 中了

JDK8 引入了 **“MetaSpace 元空间”** 的概念，专门用来存储常量、静态变量，它的内存上限是“本地内存大小”

类的信息

类加载的 **加载阶段**

- JVM 虚拟机会将类信息从 Class 文件加载进内存（方法区的运行时数据结构）
- 同时会在内存中生成一个 Class 对象（对于 HotSpot 虚拟机而言是在方法区），作为方法区访问该类数据的入口

加载类的过程，只有 **主动加载** 的时候才会加载进内存

- 主动加载：**new、反射、JNI 反射**，在方法区生成一个 Class 对象
- 非主动加载，只是在方法区创建一个 **int 类型的符号引用**，代表该类

```
public class Student {  
    // 这个时候 Tearcher 并不会加载进内存  
    // 而是先在方法区中生成一个类的符号引用  
    // 等到真正加载改类的时候，才会将 class 字节码加载进内存，解析成方法区的  
    各种结构  
    public Tearcher teacher;  
  
    public void main() {  
        teacher = new Teacher();    // 只有执行到改方法的时候，才会真  
        正加载进内存  
    }  
}
```

- 方法表

方法表 在内存中堆结构是一个 **ArtMethod** 对象

- 字段表

字段表 在内存中堆结构是一个 **ArtField** 对象

- 属性表

code 属性表：栈编译后的代码

- max_locals 属性：局部变量表的大小

- `max_stacks` 属性：栈的最大深度

常量池

运行时常量池（Runtime Constant Pool），一般存储随着类的加载而加载进方法区的常量以及符号引用

- **常量**：除了存储一些编译期产生的常量，还可以存储在运行时动态加入的常量：`String.intern()`
- **符号引用**：int 类型，例如 **类的符号引用**，如果一个类被加载过，就会在常量池中存在一个符号引用（本质还是 String）

静态变量

static 修饰的成员变量什么时候被回收？有3种情况

- 如果是一个 **基本数据类型**：static int 、 static char
 - 对象的符号引用存储在方法区的 **静态变量区**，实际的值也存储在 **静态变量区**
 - 直到程序结束，一直占有内存
- 如果是一个 **引用类型**：static Person、static Animal、static String(new String)
 - 对象的符号引用存储在方法区的 **静态变量区**，实际对象是存储在 **堆** 中的

- 我们可以将对象置空 null，来通知 GC 回收
- 如果是直接通过 **字面值常量赋值的 String 对象**
 - 对象的符号引用存储在方法区的 **静态变量区**，实际对象存储在 **常量池** 中
 - JVM 会将这个字面值常量作为常量存储在方法区的 **常量池** 中；一旦创建就不会被回收

总结

- 如果是引用类型（String的字面值常量除外），可以置空，通知 GC 回收
- 如果是一个简单类型，我们无法控制它的生命周期

直接内存

直接内存 - Direct Memory

直接内存：通过 Native 函数分配的一个 **堆外内存**，**避免 Java 堆和 Native 堆来回复制数据**

- 并不属于 Java 虚拟机规范的内存区域，但是因为会被频繁的使用，也会造成 OutOfMemoryError 异常
- 基于 Channel 通道和 Buffer 缓冲区的 I/O 流，可以为 Native 函数库直接分配一个堆外内存（直接内存）
- 然后通过 Java 堆中的 **DirectByteBuffer** 对象作为这块内

存的引用，提高性能，避免 Java 堆和 Native 堆来回复制数据

内存分配实战

// class信息存储在方法区中

// Person实例存储在堆中

```
public class Person {
```

```
    private String name = "小明";
```

// name引用跟随对

象存入堆区，name实例放入常量池

```
    private Student student = new Student();
```

// student引用跟随

对象存入堆区，student实例存入堆中

```
    private int age;
```

// age=0跟随对象直

接放入堆区

// 构造方法放入方法区（方法表），方法内代码作为 Code 属性也放入方法区（属性表）

```
    public Person(String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

// person1 是引用放入虚拟机栈区（局部变量表），new 关键字开辟堆内存
Person 自定义对象放入堆区

```
    Person person1 = new Person("张三", 18);
```

```
    Person person2 = new Person("李四", 20);
```

```

// 通过 person 引用创建 toString() 方法栈帧
person1.toString();
person2.toString();
}

}

```

方法中的局部变量：跟随方法存储在局部变量表中（栈）

- 对象的引用，存储在栈的 **局部变量表** 中
- 基本数据类型，跟随对象直接存储在 **栈** 中

类中的成员变量：跟随对象存储在堆中（堆）

- 对象的引用，跟随对象存储在 **堆** 中
- 基本数据类型，跟随对象直接存储在 **堆** 中

```

// JVM 启动时将 Test.class 放入方法区
public class Test {
    // main 方法放入方法区，方法内代码作为 Code 属性放入方法区
    public static void main(String[] args) {
        // person1 是引用放入虚拟机栈区，new 关键字开辟堆内存 Person 自定义对象放入堆区
        Person person1 = new Person("张三", 18);
        Person person2 = new Person("李四", 20);

        // 通过 person 引用创建 toString() 方法栈帧
        person1.toString();
        person2.toString();
    }
}

```

// JVM 启动时将 Person.class 放入方法区

```
public class Person {
```

// new Person 创建实例后, name 引用放入堆区, name 实例对象放入常量池

```
    private String name;
```

// new Person 创建实例后, age = 0 直接放入堆区

```
    private int age;
```

// Person 方法放入方法区, 方法内代码作为 Code 属性放入方法区

```
    public Person(String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

// toString 方法放入方法区, 方法内代码作为 Code 属性放入方法区

```
    @Override
```

```
    public String toString() {
```

```
        return "Person{" + "name='" + name + '\'' + ", age="
```

```
+ age + '\'';
```

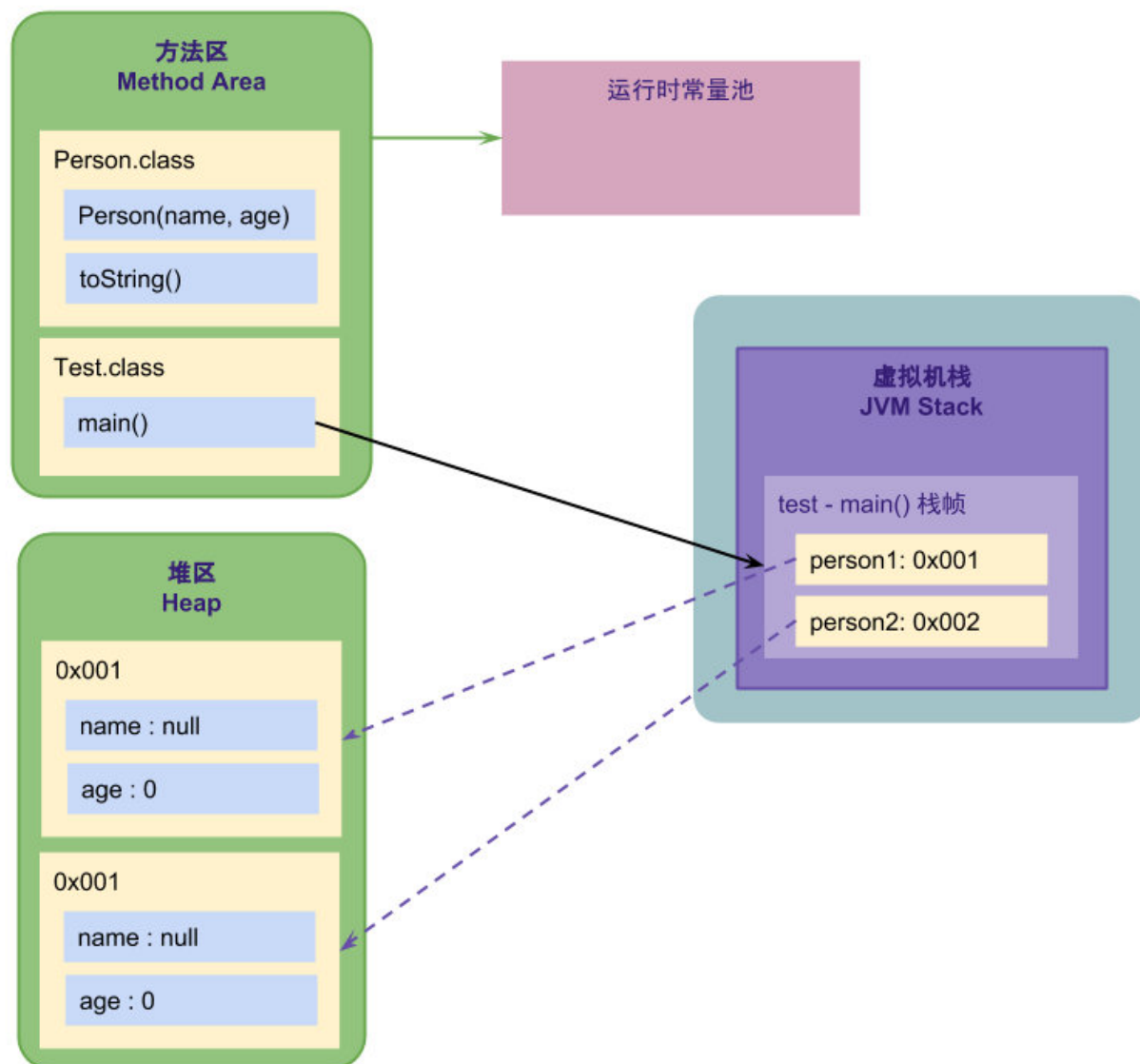
```
    }
```

```
}
```

1. 首先 JVM 会将 Test.class, Person.class 加载到方法区, 找到有 main() 方法的类开始执行
 - 加载类信息到方法区: 包括 Class 对象、属性表、方法表 (代码存储在 code 属性中)
 - 为每个 Java 方法创建一个栈帧, 并存入虚拟机栈中
2. 当执行到 Person person1 = new Person("张三", 18) 这段代

码的时候

- 创建的实例对象放入堆中
- 对象的 引用 放入虚拟机栈的 **局部变量表** 中

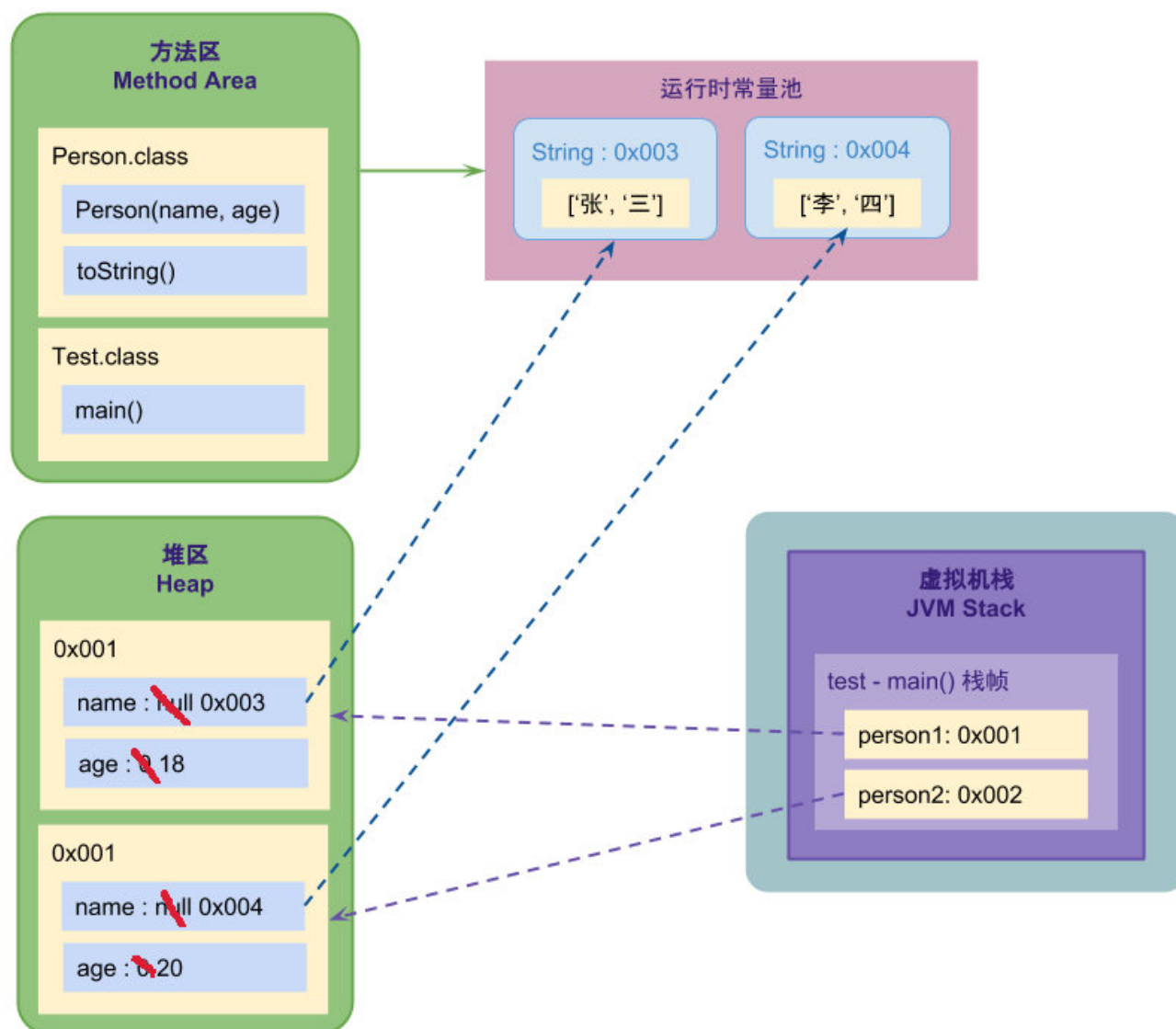


3. 接着会调用 Person 的构造方法，进行赋值

- Person 中的字段，**字段的引用跟随 Person 存储在堆中**
- 字段的实例对象，根据不同类型存储在不同区域
 - 字面值常量存储在 **常量池** 中
 - 基本数据类型直接跟随对象存储在 **堆** 中

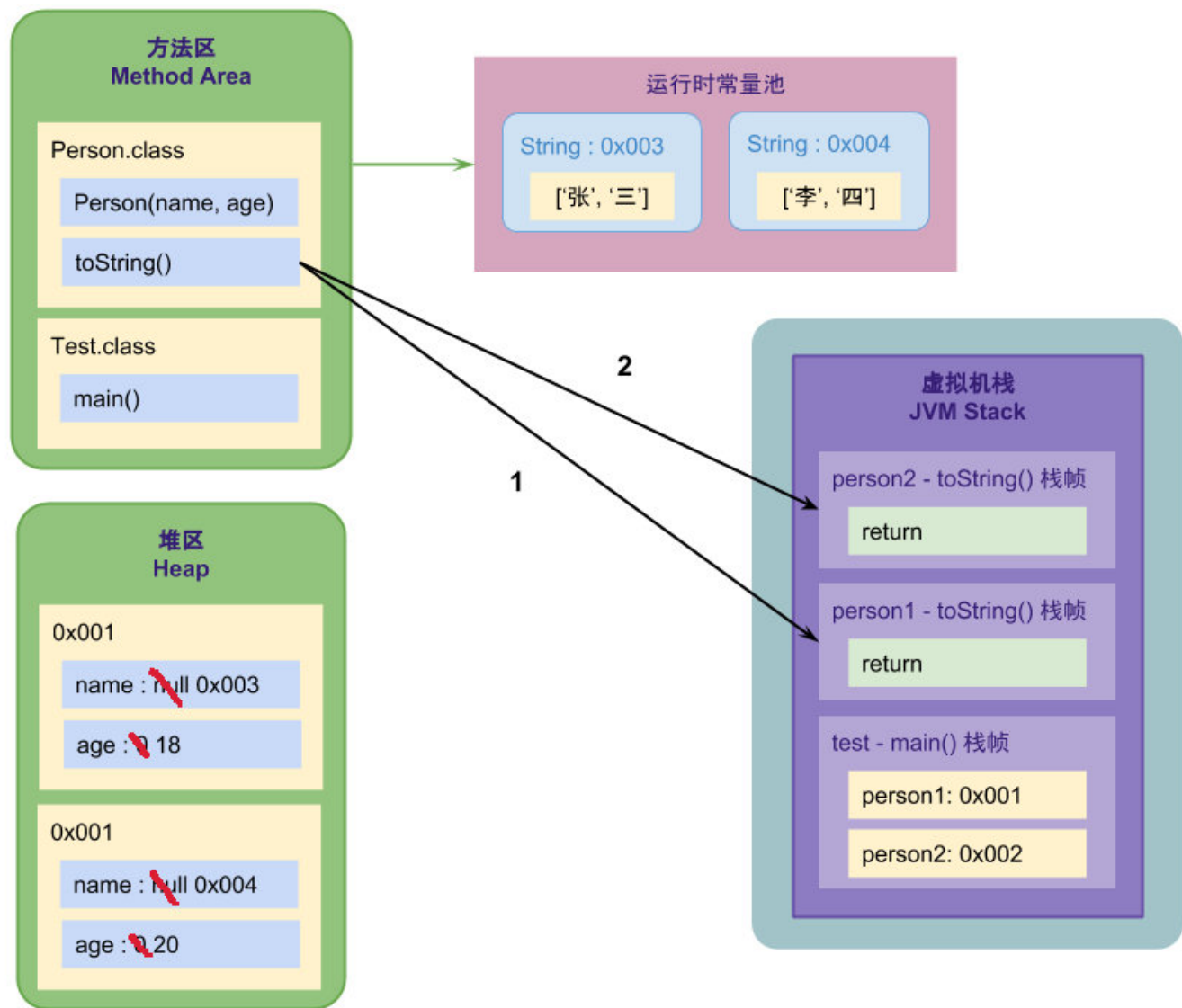
- 引用类型存储在 **堆** 中

- 类中的字段（类变量）默认有一个初始值，局部变量没有初始值



4. `Person` 对象实例化之后，接着会调用 `toString()` 方法

- 同 `main()` 方法一样 JVM 会创建一个 `toString()` 的栈帧放入虚拟机栈中，执行完之后返回一个值



Android 内存模型

通常一个应用对应一个 JVM 虚拟机（一个进程），系统默认会给每个应用分配固定的最大可使用内存，一旦超出这个值，就会报 OOM 异常

- 获取内存的使用状况，可用借助 `ActivityManager`、

Runtime 获取

- 早期只分配了 16M、24M 大小，现在则分配了 256M、512M 大小，不同手机略有不同
- 如果我们在清单文件中配置了 application 的 `android:largeHeap="true"` 属性，那么表示允许该应用在更大的 Dalvik 堆中获取更大的内存

// 查看最大内存空间

```
adb shell getprop dalvik.vm.heapsize
```

// 使用 AM 查看

```
ActivityManager activityManager = (ActivityManager)
getSystemService(Context.ACTIVITY_SERVICE);
```

```
int largeMemoryClass = activityManager.getLargeMemoryClass();
```

// 始终为 largeHeap 的值

// 使用 Runtime 查看

```
long maxMemory = Runtime.getRuntime().maxMemory();
```

// 跟随 largeHeap=true/false 而改变

```
long totalMemory = Runtime.getRuntime().totalMemory();
```

```
long freeMemory = Runtime.getRuntime().freeMemory();
```

```
Log.d(TAG, "onCreate: 最大内存" + maxMemory + " 最大内存" +
totalMemory + " 可用内存" + freeMemory);
```